PrivacyGNN Beginner Guide

Learn the whole project from zero, in plain English

Prepared for Krish Sharma

May 11, 2026

Abstract

This guide explains your PrivacyGNN project as if you are learning it for the first time. It avoids assuming that you already know graph neural networks, privacy attacks, experiment pipelines, or the codebase. The goal is simple: after reading this PDF, you should understand what the project is trying to prove, how the code works, what every major file does, what the results mean, and how to explain the project to someone else.

Contents

1 The Project in One Sentence

Your project asks:

> **If a graph neural network is trained on some nodes in a graph, can an attacker look at the model's prediction scores and guess which nodes were used for training?**

That is the whole project. Everything else in the repository exists to answer that question carefully.

More simply:

- You train models on graph data.
- You ask the model to predict labels.
- You look at the model's confidence scores.
- You test whether those scores reveal who was in the training set.
- You try simple defenses to reduce that leakage.

2 A Very Simple Example

Imagine a classroom.

- A teacher studies 50 students for a test.
- Later, someone asks the teacher about 100 students.
- For each student, the teacher gives an answer and says how confident they are.
- If the teacher is much more confident about the 50 students they studied, an outsider may guess who was in the study group.

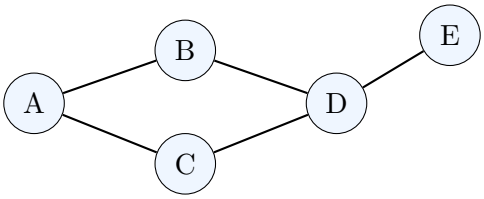
In your project:

- the **teacher** is the machine learning model,
- the **students** are graph nodes,
- the **study group** is the training set,
- the **outsider** is the privacy attack,
- the **confidence** is the model's predicted probability.

> **Membership inference** means guessing whether an example was a member of the training set.

3 What Is a Graph?

A **graph** is just a set of things connected by lines.



The circles are called **nodes**. The lines are called **edges**.

Examples of graphs:

- people connected by friendships,
- papers connected by citations,
- computers connected by network traffic,
- patients connected by similarity,
- molecules connected by chemical bonds.

In your project, each node has:

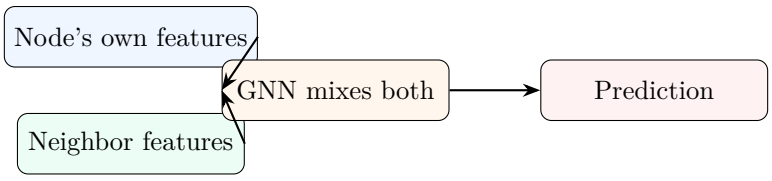
- **features**: numbers describing the node,
- **label**: the correct category,
- **neighbors**: other nodes connected to it.

4 What Is a GNN?

A **graph neural network**, or GNN, is a model that learns from both:

- a node's own features,
- the features of nearby nodes.

Think of it like asking your friends for advice. A normal model only looks at you. A GNN looks at you and your neighborhood.



Your project compares:

- **LogReg**: simple model, ignores graph edges.
- **MLP**: neural network, ignores graph edges.
- **GCN**: graph neural network, uses neighbors.
- **GraphSAGE**: another graph neural network, uses neighbors differently.

5 What Is Privacy Leakage?

Privacy leakage means the model accidentally reveals something it should not.

Here, the secret is:

Was this node part of the model's training data?

The attacker does not need to see the training set. The attacker only looks at the model's output probabilities.

For example, suppose the model outputs:

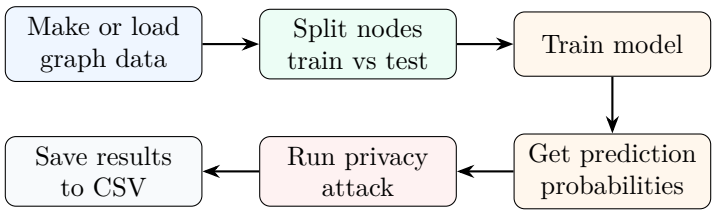
$$[0.01,\ 0.02,\ 0.95,\ 0.01,\ 0.01]$$

That means the model is very confident in class 3.

If the model is usually more confident on training nodes than test nodes, then confidence becomes a clue. The attack uses that clue.

6 The Main Pipeline

Here is the whole project as a simple flow:



Every Python file supports one part of this flow.

7 The Four Research Questions

Your project is organized around four questions:

1. **Do GNNs leak more privacy than non-graph models?**
   This compares GCN/GraphSAGE against LogReg/MLP.
2. **Does graph structure affect leakage?**
   This studies homophily and density.
3. **Do simple defenses reduce leakage?**
   This tests DropEdge, label smoothing, early stopping, confidence masking, and edge sparsification.
4. **What is the accuracy vs privacy tradeoff?**
   A good model should be accurate but not leak membership information.

8 Two Graph Words You Must Know

8.1 Homophily

Homophily means similar nodes connect to similar nodes.

Example:

- High homophily: computer science papers cite mostly computer science papers.
- Low homophily: computer science papers cite many unrelated fields.

In the code, homophily means:

$$\frac{\text{edges connecting same-label nodes}}{\text{all edges}}$$

8.2 Density

Density means how many edges the graph has.

- Sparse graph: few connections.
- Dense graph: many connections.

Your synthetic graphs test:

- high homophily vs low homophily,
- sparse vs medium vs dense.

That creates six synthetic settings.

9 What Data Does the Project Use?

The project uses two kinds of data.

9.1 Real Citation Graphs

- **Cora**
- **Citeseer**

In these datasets:

- nodes are research papers,
- edges are citation links,
- labels are paper topics,
- features describe words in the paper.

9.2 Synthetic Graphs

Synthetic means the project creates fake graphs itself. This is useful because you can control the graph.

The six synthetic datasets are:

- `synthetic_high_sparse`
- `synthetic_high_medium`
- `synthetic_high_dense`
- `synthetic_low_sparse`
- `synthetic_low_medium`
- `synthetic_low_dense`

The name tells you the setting:

`synthetic_high_sparse` = high homophily + sparse graph.

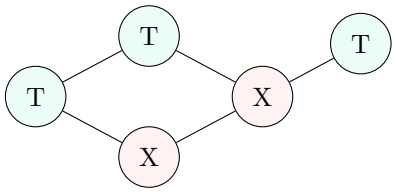
10 What Is Training?

Training means showing the model examples so it can learn.

In your project:

- some nodes are training nodes,
- some nodes are test nodes,
- the model learns from training nodes,
- the model is evaluated on test nodes.

The privacy problem comes from this difference. If the model behaves differently on training nodes than test nodes, an attacker may detect it.



T = training node, X = test node

11 What Are the Defenses?

Defenses are attempts to make the model leak less.

Defense	Beginner explanation
No defense	Train normally. This is the baseline.
DropEdge	Randomly remove some graph edges while training, so the model cannot rely too heavily on exact graph connections.
Label smoothing	Do not train the model with completely hard labels. Instead of saying "100 percent class A", say something softer like "90 percent class A, 10 percent spread out".
Early stopping	Stop training early so the model does not memorize too much.
Confidence masking	After prediction, hide some probability details by keeping only the top few class scores.
Edge sparsification	Remove some edges before training to make the graph simpler.
DP-SGD	A formal privacy idea listed in config, but not actually active in this checkout because the needed minibatch code is a stub.

12 What Are the Attacks?

An attack tries to guess:

training node or test node?

12.1 Confidence Attack

This attack looks at how confident the model is.

If the model says:

99% confident

the attack may think:

Maybe this was a training node.

12.2 Threshold Attack

This is an even simpler attack. It uses one rule:

> If confidence is above some cutoff, guess "member". Otherwise guess "non-member".

12.3 Shadow Attack

A shadow attack trains a fake copy of the situation.

Beginner analogy:

- If you want to understand how a teacher behaves, you train another teacher in a similar classroom.
- You study that second teacher's behavior.
- Then you use what you learned to attack the real teacher.

In code:

- train a shadow model,
- learn what member vs non-member outputs look like,
- attack the target model.

12.4 LiRA

LiRA is listed, but in this checkout it is not really implemented. The file returns random-guess values:

$$0.5,\ 0.5$$

So do not treat LiRA results as real unless that code is replaced.

13 How to Read Attack AUC

AUC is a score for how well the attack separates members from non-members.

Attack AUC	Meaning
0.5	Random guessing. Good privacy.
0.6	The attack has some signal. Some leakage.
0.7	The attack is doing clearly better than random. More leakage.
1.0	Perfect attack. Very bad privacy.

For this project:

lower attack AUC is better for privacy.

For accuracy:

higher accuracy is better for usefulness.

So the best situation is:

high accuracy and attack AUC near 0.5.

14 The Codebase in Plain English

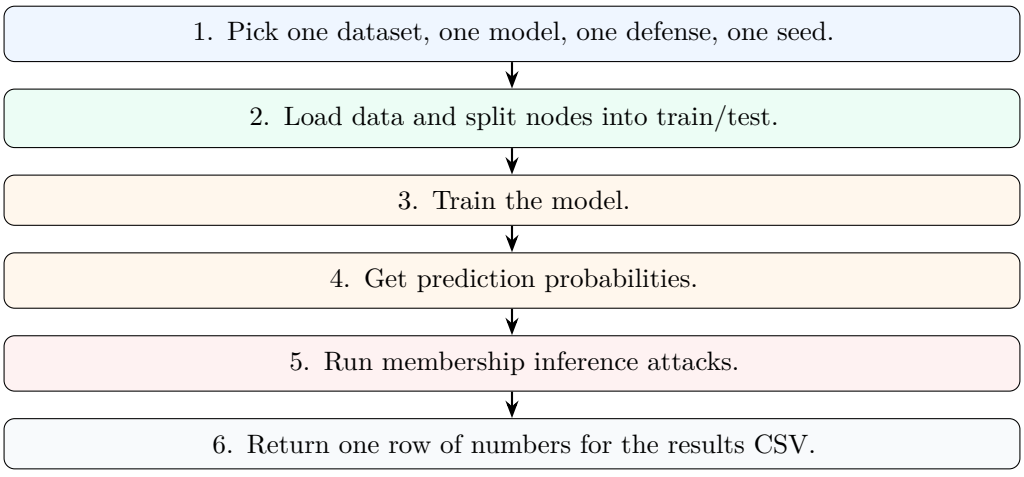
File	What it does in beginner words
`config.py`	Reads the experiment settings and creates the list of all runs. It is like the schedule for the experiment.
`experiment_config.yaml`	The main settings file. It says which datasets, models, defenses, attacks, and seeds to run.
`experiment_config_paper.yaml`	A smaller, paper-safe settings file. Good for reproducing paper figures.
`data.py`	Loads real graph data and creates synthetic graph data. Also measures homophily and density.
`models.py`	Defines the GCN and GraphSAGE neural network models.

File	What it does in beginner words
`training.py`	Teaches the GNN models using training nodes. Also applies some defenses.
`attacks.py`	Contains the privacy attacks and calibration calculation.
`experiment.py`	The most important file. Runs one complete experiment from data to model to attacks to one result row.
`run_final.py`	Runs all experiments from the config and saves CSV files.
`stats_utils.py`	Compares defenses statistically and creates bootstrap confidence intervals.
`generate_figures.py`	Makes publication-style figures from results.
`generate_basic_figures.py`	Makes simpler figures.
`generate_final_visuals.py`	Makes polished final presentation visuals.
`generate_poster_visuals.py`	Makes many poster-ready visuals.
`fix_figures.py`	Fixes two figures so labels look better.
`ogb_loader.py`	Placeholder for large graph datasets. Not active now.
`graph_minibatch.py`	Placeholder for large graph minibatch training and DP-SGD. Not active now.
`lira_attack.py`	Placeholder for LiRA. Returns random-guess results.
`run_everything_final.sh`	Runs experiments and figures in one command.

15 The Most Important File: experiment.py

If you only study one file deeply, study `experiment.py`.

It contains `run_one`, which performs one complete experiment:



`run_final.py` simply calls this many times.

16 What Results Files Mean

Result file	Beginner meaning
`all_results.csv`	The big table. Every row is one experiment run.
`summary.csv`	Averages results across seeds so they are easier to read.
`significance.csv`	Checks whether each defense is meaningfully different from no defense.
`significance_lira.csv`	Same idea for LiRA, but LiRA is placeholder here.
`summary_bootstrap.csv`	Gives uncertainty ranges around average results.
`all_results.partial.csv`	A checkpoint saved while experiments are still running.

17 What the Main Columns Mean

Column	Beginner meaning
`dataset`	Which graph was used.
`model`	Which model was trained.
`defense`	Which privacy defense was used.
`seed`	Random seed, used so experiments can be repeated.
`homophily`	How often connected nodes have the same label.
`density`	How connected the graph is.
`test_accuracy`	How often the model predicts the correct label on test nodes.
`test_f1`	Another accuracy-like score, useful when classes are uneven.
`test_auroc`	A classification quality score.
`conf_attack_auc`	How well the confidence attack guesses membership. Lower is better privacy.
`threshold_attack_auc`	How well the simple threshold attack works.
`shadow_attack_auc`	How well the shadow-model attack works.
`lira_attack_auc`	Placeholder in this checkout.
`ece_test`	Calibration error. Lower means confidence is more honest.
`dp_epsilon`	Differential privacy budget if DP-SGD ran. Usually blank here.

18 What Libraries Are Used?

Library	Simple explanation
PyTorch	Main deep learning library. Used for tensors, neural networks, and training.
PyTorch Geometric	Graph machine learning library. Gives graph data objects and GNN layers.
NumPy	Basic numerical computing and random number generation.
scikit-learn	Logistic regression, MLP baseline, attack classifier, and metrics.
pandas	Reads and writes CSV files, groups results, computes summaries.

Library	Simple explanation
SciPy	Statistical tests.
matplotlib / seaborn	Draws figures and heatmaps.
PyYAML	Reads YAML config files.
ReportLab	Builds one manuscript PDF from Python.
adjustText	Helps figure labels not overlap.
OGB / Opacus	Listed for future large-graph and DP work, but not active in this checkout.

19 What Is Implemented and What Is Not

19.1 Implemented

- Cora and Citeseer loading through PyTorch Geometric.
- Synthetic graph generation with controlled homophily and density.
- LogReg, MLP, GCN, and GraphSAGE experiments.
- Full-batch GNN training.
- Confidence, threshold, and shadow attacks.
- Defense comparisons.
- CSV summaries, significance tests, and bootstrap intervals.
- Multiple figure-generation scripts.

19.2 Not Really Implemented Yet

- Real LiRA attack.
- OGB/Reddit large graph loading.
- Neighbor-sampled minibatch training.
- Working DP-SGD path.
- Automated unit test suite.

This is not bad. It just means you should be honest when explaining the project.

20 How to Explain the Project Out Loud

Here is a beginner-friendly explanation you can say:

This project studies privacy leakage in graph neural networks. I train models on graph node classification tasks, then test whether an attacker can guess which nodes were used for training by looking at the model's prediction confidence. I compare graph models against non-graph baselines, test different graph structures like homophily and density, and evaluate lightweight defenses such as DropEdge, label smoothing, early stopping, confidence masking, and edge sparsification. The experiments are config-driven and save reproducible CSV results, summaries, statistical tests, and figures.

21 How to Study This Project Step by Step

Do not start with the deepest code. Use this order:

1. Read this beginner guide.
2. Read `README.md`.
3. Open `experiment_config_paper.yaml`. Understand the experiment choices.
4. Open `run_final.py`. Understand the outer loop.
5. Open `experiment.py`. Understand one run.
6. Open `data.py`. Understand synthetic graph creation.
7. Open `models.py`. Understand GCN and GraphSAGE.
8. Open `training.py`. Understand training and defenses.
9. Open `attacks.py`. Understand attack AUC.
10. Open `results/summary.csv`. Connect code to numbers.
11. Open figure scripts last.

22 Command Cheat Sheet

```
cd privacy-gnn

# Install dependencies
pip install -r requirements.txt

# Run the default experiment config
python run_final.py

# Run the paper-safe config
PRIVACYGNN_CONFIG=experiment_config_paper.yaml python run_final.py

# Make figures
python generate_figures.py

# Run experiments and figures together
bash run_everything_final.sh
```

23 Tiny Glossary

Word	Plain meaning
Node	One item in a graph. Example: one paper or one person.
Edge	A connection between two nodes.
Feature	Numbers describing a node.
Label	The correct category for a node.
Training set	Data the model learns from.
Test set	Data held back to check if the model generalizes.
GNN	A neural network that uses graph connections.
GCN	A common GNN model.
GraphSAGE	Another GNN model.
MIA	Membership inference attack. Guessing if data was in training.
AUC	Attack quality score. 0.5 means random guessing.
Defense	A technique to reduce leakage.
Homophily	Connected nodes tend to have the same label.
Density	How many edges the graph has.
ECE	Calibration error. Measures if confidence is honest.
Seed	A number that controls randomness so runs can be repeated.

24 Final Takeaway

Your project is not just training a graph neural network. It is measuring whether graph models reveal training membership, testing when that leakage happens, and checking whether simple defenses help.

If you understand the flow:

$$\text{data} \rightarrow \text{model} \rightarrow \text{predictions} \rightarrow \text{privacy attack} \rightarrow \text{results},$$

then you understand the backbone of the whole project.